```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    classification_report
)
df = pd.read_csv("diabetes.csv")
print("Dataset loaded successfully!")
print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset Information:")
print(df.info())
print("\nMissing values:")
print(df.isnull().sum())
print("\nOutcome counts:")
print(df["Outcome"].value_counts())
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
print("Features shape:", X.shape)
print("Target shape:", y.shape)
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
   X, y,
   test_size=0.2,
   random_state=42
)
plt.figure(figsize=(12, 6))

sns.boxplot(data=X_train)

plt.title("Outlier Detection Using Boxplot")
plt.xticks(rotation=45)
plt.show()
Q1 = X_train.quantile(0.25)
Q3 = X_train.quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = (
   (X_train < lower) |
   (X_train > upper)
)

print("Outliers in each feature:")
print(outliers.sum())
columns = [
   'Glucose',
```

```
    'BloodPressure',
    'SkinThickness',
    'Insulin',
    'BMI'
]

for col in columns:
    print(col, ":", (df[col] == 0).sum())
columns = [
    'Glucose',
    'BloodPressure',
    'SkinThickness',
    'Insulin',
    'BMI'
]

# Replace invalid zeros with NaN
df[columns] = df[columns].replace(0, np.nan)

# Check missing values
print("Missing values:")
print(df[columns].isnull().sum())

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
```

```
)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("Feature scaling completed!")
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
print("KNN model trained successfully!")
y_pred = knn.predict(X_test)
print("Prediction completed!")
print("Predictions:")
print(y_pred)
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(6, 4))
sns.heatmap(
  cm,
  annot=True,
  fmt="d",
  xticklabels=["No Diabetes", "Diabetes"],
  yticklabels=["No Diabetes", "Diabetes"]
)
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix")
plt.show()
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Accuracy (%):", accuracy * 100)
error_rate = 1 - accuracy

print("Error Rate:", error_rate)
print("Error Rate (%):", error_rate * 100)
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_pred)
print("Precision:", precision)
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_pred)
print("Recall:", recall)
print("\n==============================")
print("KNN PERFORMANCE")
print("==============================")
print("Confusion Matrix:")
print(cm)
print("\nAccuracy  :", accuracy)
print("Error Rate :", error_rate)
print("Precision  :", precision)
print("Recall    :", recall)

from sklearn.metrics import classification_report
print("\nClassification Report:")
print(classification_report(
  y_test,
```

```
    y_pred,
    target_names=["No Diabetes", "Diabetes"]
))
```

K-Nearest Neighbours (KNN) is a supervised machine learning algorithm that predicts the class or value of new data by finding the K closest data points in the existing dataset.

Example: Predicting whether a person likes a movie

Suppose a new user likes action movies. We want to predict whether they will like a particular movie.

KNN works as follows:

1. Choose K: Select the number of neighbours, for example K = 3.
2. Calculate Distance: Calculate the distance between the new data point and existing data points.
3. Find Nearest Neighbours: Select the 3 closest data points.
4. Majority Voting: Check which category is most common among those 3 neighbours.
5. Prediction: Assign the new data point to the majority category.

Macro average calculates the average of a metric (precision, recall or F1-score) for all classes by giving equal importance to each class.

Weighted average calculates the average of metrics by considering the number of samples (support) in each class.

Your support values are:

- No Diabetes = 100
- Diabetes = 54
- Total = 154

The formula is:

$$ Weighted\ Average = \frac{(Metric_1 \times Support_1)+(Metric_2 \times Support_2)}{Total} $$

For example, weighted precision:

$$ =\frac{(0.75\times100)+(0.58\times54)}{154} $$

$$ \approx 0.69 $$

 Accuracy (70.13%): Moderate starting performance.

 Precision (58.33%): Of the patients predicted as diabetic, around 58% actually have diabetes.

 Recall (51.85%): Your model identifies only around 52% of actual diabetic patients.

 F1-score (55%): The balance between precision and recall needs improvement.

Actual / Predicted	No Diabetes	Diabetes
No Diabetes	80	20
Diabetes	26	28

This means:

- 80 patients correctly classified as No Diabetes.
- 20 patients incorrectly classified as Diabetes.
- 26 diabetic patients incorrectly classified as No Diabetes.
- 28 diabetic patients correctly classified as Diabetes

How can you improve your KNN model?

I recommend trying these improvements:

1. Experiment with different K values (3, 5, 7, 9, 11).
2. Try StandardScaler and RobustScaler.
3. Try different distance metrics.
4. Use stratified cross-validation.
5. Compare KNN with SVM and Random Forest.

KNN model achieved 70.13% accuracy. It gives moderate performance, but diabetic-class recall is 51.85%, so further improvement is required. Which metric is important in diabetes prediction?

Recall is particularly important because we want to identify as many actual diabetic patients as possible.

KNN is used in various applications such as disease prediction, movie recommendation, product recommendation, handwriting recognition, house price prediction, fraud detection and image classification. It works by comparing new data with its nearest existing data points